CRAB: A NOVEL CLUSTERING SCORE USING CLUSTERING WITH RIVALS AND BUDDIES FOR UNSUPERVISED LEARNING

A Thesis

presented to

the Faculty of California Polytechnic State University,

San Luis Obispo

In Partial Fulfillment

of the Requirements for the Degree

Master of Science in Statistics

by

Allen Choi

June 2026

© 2026

Allen Choi

ALL RIGHTS RESERVED

COMMITTEE MEMBERSHIP

TITLE: cRab: A Novel Clustering Score using Clustering with Rivals and Buddies for Unsupervised Learning

AUTHOR: Allen Choi

DATE SUBMITTED: June 2026

COMMITTEE CHAIR: Kelly Bodwin, Ph.D.
Thesis Advisor

COMMITTEE MEMBER: Alexander Dekhtyar, Ph.D.
Professor of Statistics

COMMITTEE MEMBER: Kevin Ross, Ph.D.
Professor of Statistics

ABSTRACT

cRab: A Novel Clustering Score using Clustering with Rivals and Buddies for Unsupervised Learning

Allen Choi

We present a novel clustering score approach relying on a method that classifies pairs of points as either ”rivals” or ”buddies”, constructing a score for a given number of clusters. We then present results demonstrating the properties of this metric on datasets with no clusters.

Keywords: Clustering. Unsupervised Clustering. Score. Machine Learning.

ACKNOWLEDGMENTS

I'd like to first thank my amazing Mom and Dad for providing me so much emotional support over these past half a decade at Cal Poly. Without them, I don't know how I would have gotten through college.

But without a doubt, in regards to my statistical knowledge and skills I have no one else to thank but the amazing Statistics faculty at Cal Poly, who have always been patient, understanding, and kind.

Also a thank you to Andrew Kerr for providing a quarto template: https://github.com/Wander03/CalPolySTAT-thesis-template.

TABLE OF CONTENTS

Page

LIST OF TABLES vii

LIST OF FIGURES viii

CHAPTER

1 The Problem and What's been Tried 1

1.1 Motivation 1

1.1.1 Elbow Method 1

1.1.2 Silhouette Score 3

1.2 Overview of Unsupervised Clustering Methods 4

1.2.1 k-Means Clustering 5

1.2.2 Hierarchical Clustering 5

1.3 Literature Review 5

2 Details 6

2.1 Calculated 6

2.2 Sensitivity 6

2.3 Known Data 6

2.4 Performance on penguins dataset 6

3 Computational Details 9

3.1 Matrix Stacking 9

3.2 Algorithm Construction 10

3.3 Score Construction 10

3.4 Point Contribution 13

3.5 Speed and Memory Scaling 13

3.5.1 Scaling with the Number of Observations 13

4 Null Distribution 18

4.1 Null Distribution of Scores 18
4.2 Null Distribution over Resamples 19
4.3 Adj-CRAB Score 19
5 Conclusion 20
REFERENCES 21
APPENDICES
A Resample Matrix 23
B Mean Matrix 27
C Metric Calculation 28
D Point Contribution Function 29
E Get Point Vector Helper 30
F CRAB Score Across k Clusters 31

LIST OF TABLES

Table Page

4.1 Variances for 20% Resample Calculation 19

LIST OF FIGURES

Figure Page

1.1 Scatterplot of the Example Dataset 2

1.2 WCSS as Number of Clusters k Increases 3

1.3 Silhouette Score as Number of Clusters k Increases 4

3.1 Plot of Penguin Species by Bill and Flipper Length 10

3.2 The distribution of values inside the mean matrix after running the algorithm on the penguins dataset, using K-Means and k = 2 11

3.3 The distribution of values inside the mean matrix after running the algorithm on the penguins dataset, using K-Means and k = 3 12

3.4 Speed of the CRAB Algorithm Quadratically Increases as Number of Observations Increases 14

3.5 Memory of the CRAB Algorithm Quadratically Increases as Number of Observations Increases 15

3.6 Speed of the CRAB Algorithm Increasing Linearly as Number of Dimensions Increases 16

3.7 Memory of the CRAB Algorithm Increasing Linearly as Number of Dimensions Increases 17

4.1 Distribution of Scores on a Random p-Dimensional Normal Distribution, Faceted by Resample % 18

Chapter 1

THE PROBLEM AND WHAT'S BEEN TRIED

1.1 Motivation

Unsupervised clustering algorithms such as K-Means and hierarchical clustering are commonly used today when the number of clusters in a given dataset is not known beforehand, differentiating it from supervised clustering. Kodinariya (2013) presents the most common approaches used, such as the elbow method and Silhouette score. However, even with these scores there are often disagreements and caveats that we will further explore.

1.1.1 Elbow Method

The elbow method (Thorndike, 1953) is a relatively simple technique that utilizes the plotting of the sum of the Within Cluster Sum of Squares (WCSS) across all clusters at the y axis, against the number of clusters on the x-axis. Similar in behavior to the R^2 measure, adding more clusters necessarily decreases the WCSS due to the fact that adding an extra cluster will mean that at least one observation is closer to that cluster than the others. The question of "how many clusters" is then answered by looking at where this drop-off in WCSS is significant. This method however comes with many problems (Schubert, 2023). The biggest issue is that while for some datasets this "drop-off" in WCSS is easy to visually recognize, sometimes it can be extremely difficult to tell what is considered significant. In fact, (Schubert, 2023) takes a strong approach on this, where they state "Educators should omit the method or at least explain better alternatives".

We demonstrate this below with an example dataset, where

$$x_1, y_1 \sim gamma(\alpha = 1)$$

$$x_2, y_2 \sim gamma(\alpha = 1.5) + 2$$

$$x_3, y_3 \sim gamma(\alpha = 2) + 4$$

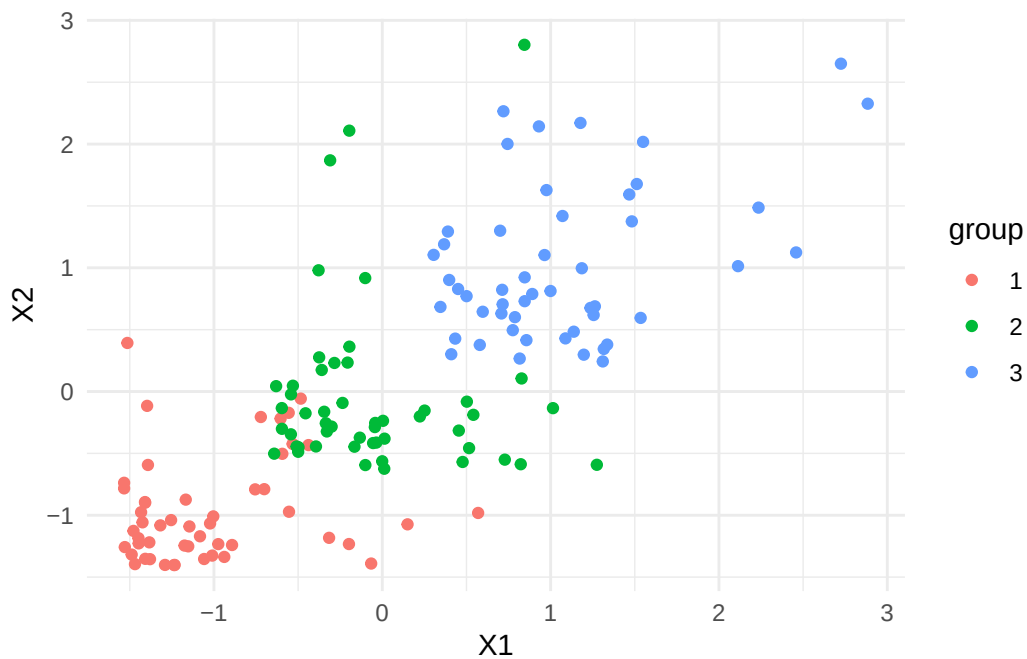


Figure 1.1: Scatterplot of the Example Dataset

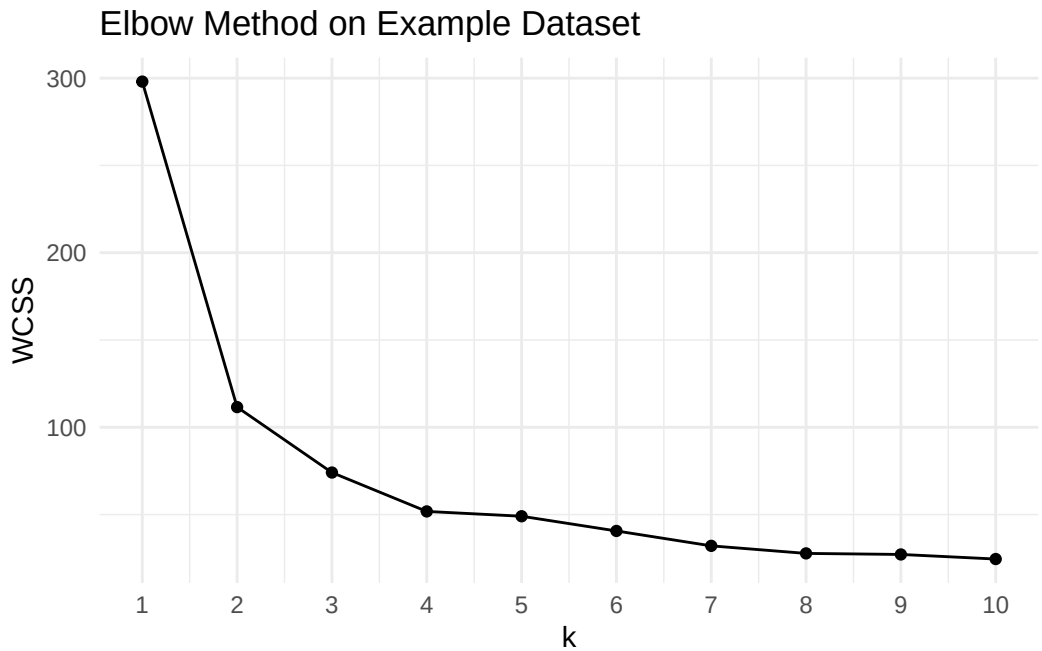


Figure 1.2: WCSS as Number of Clusters k Increases

1.1.2 Silhouette Score

The Silhouette Score approach (Rousseeuw, 1987) is a method proposed to measure how comfortably each observation belongs to its assigned cluster compared to the other possible clusters. As it is defined in the original 1986 paper, the silhouette width $s(i)$ for observation i is defined by:

$$s(i) = \frac{b(i) - a(i)}{max\{a(i), b(i)\}}$$

where $a(i)$ is defined by the average distance between the i and all other observations in the same cluster, and $b(i)$ is defined by the smallest average distance between i and all other observations in each other respective clusters.

Averaging this score gives a measure of how well a certain number of k-Clusters fits the data, similar to the elbow method. Unlike in the elbow method however, where increasing the number of clusters necessarily decreases the WCSS, the average Silhouette width does

not necessarily increase as the number of clusters increases.

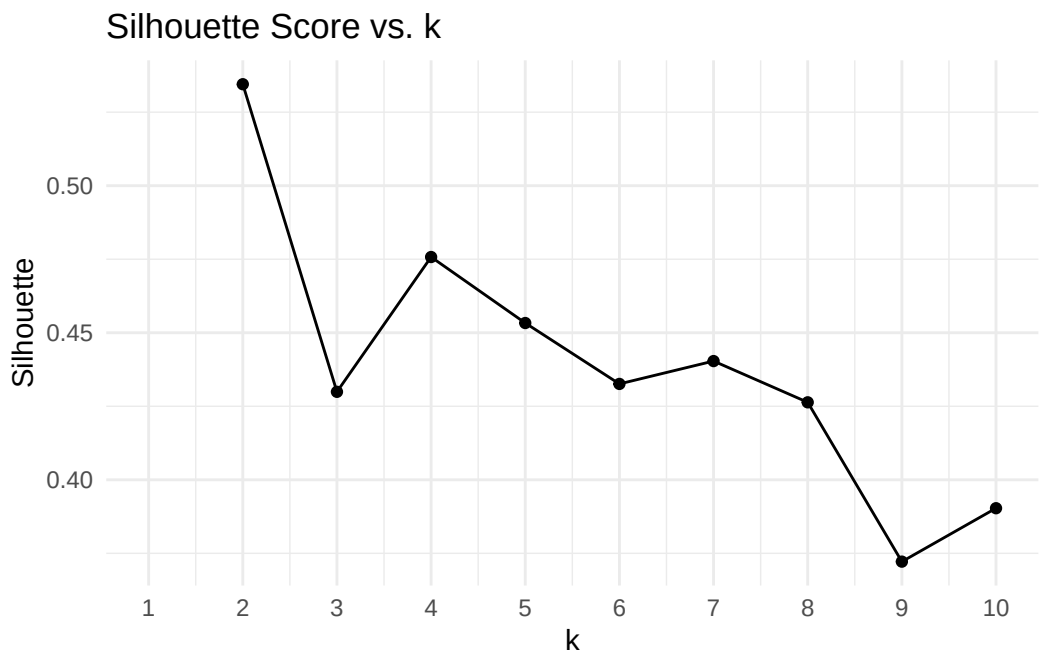


Figure 1.3: Silhouette Score as Number of Clusters k Increases

We present the CRAB algorithm, which utilizes random subsampling of a dataset in order to determine the optimal number of clusters for an unsupervised clustering algorithm.

(R Core Team, 2025). (Ben-Hur, Elisseeff, & Guyon, 2001). (Kuhn & Wickham, 2020). (Hvitfeldt & Bodwin, 2025). (Wickham et al., 2019).

1.2 Overview of Unsupervised Clustering Methods

For the scope of this paper we will go over only a few of the basic unsupervised clustering methods commonly used due to the fact that the CRAB algorithm currently only supports either k-Means or hierarchical clustering. However, in theory the CRAB algorithm framework allows it to work on any unsupervised method as long as the algorithm outputs a cluster assignment for every observation in the dataset. Note that this means that algorithms such as DBSCAN (Density-Based Spatial Clustering of Applications with Noise) will not work

with this framework due to the fact that some observations are classified as “noise” (Ester, Kriegel, Sander, & Xu, 1996).

1.2.1 k-Means Clustering

(Ikotun, Ezugwu, Abualigah, Abuhaija, & Heming, 2023)

1.2.2 Hierarchical Clustering

(Jaeger & Banks, 2023)

1.3 Literature Review

(previous work done on the resampling method) (Ben-Hur et al., 2001)

Chapter 2

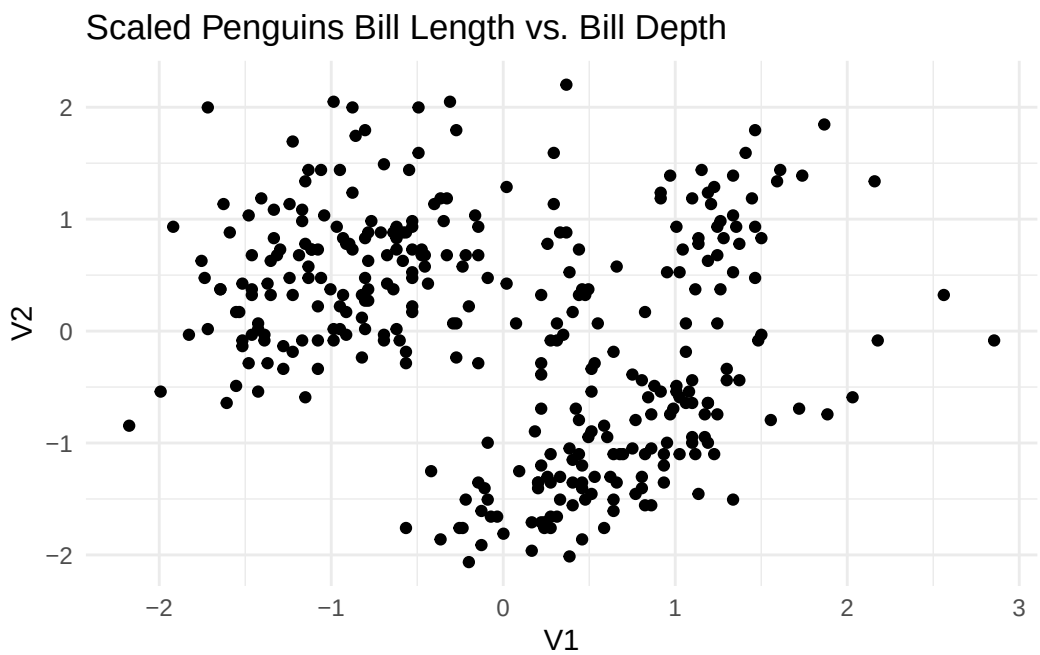
DETAILS

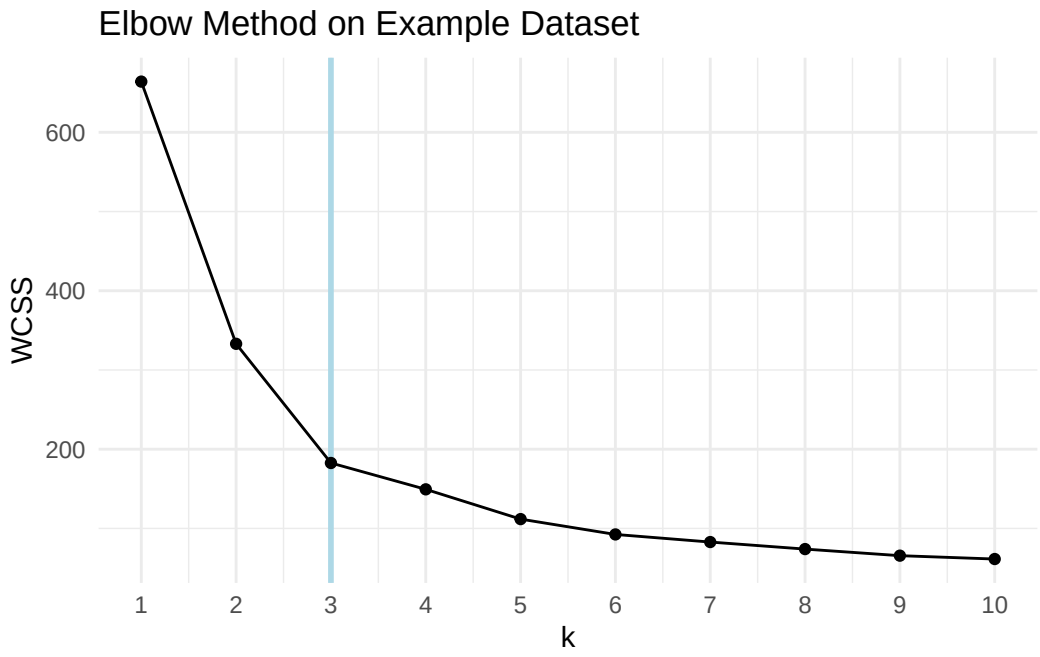
2.1 Calculated

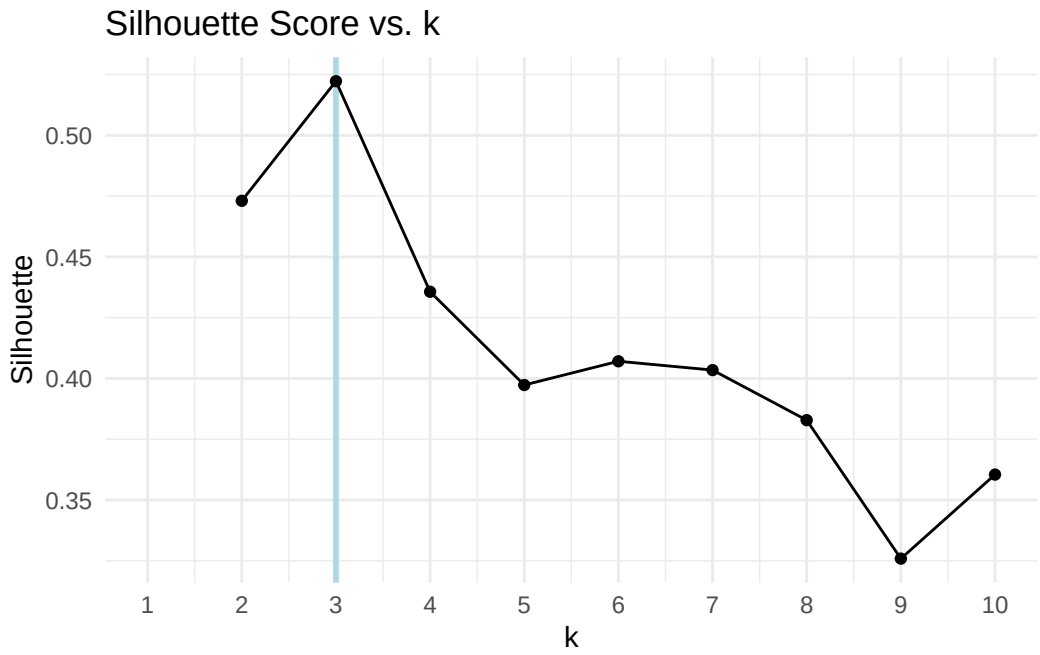
2.2 Sensitivity

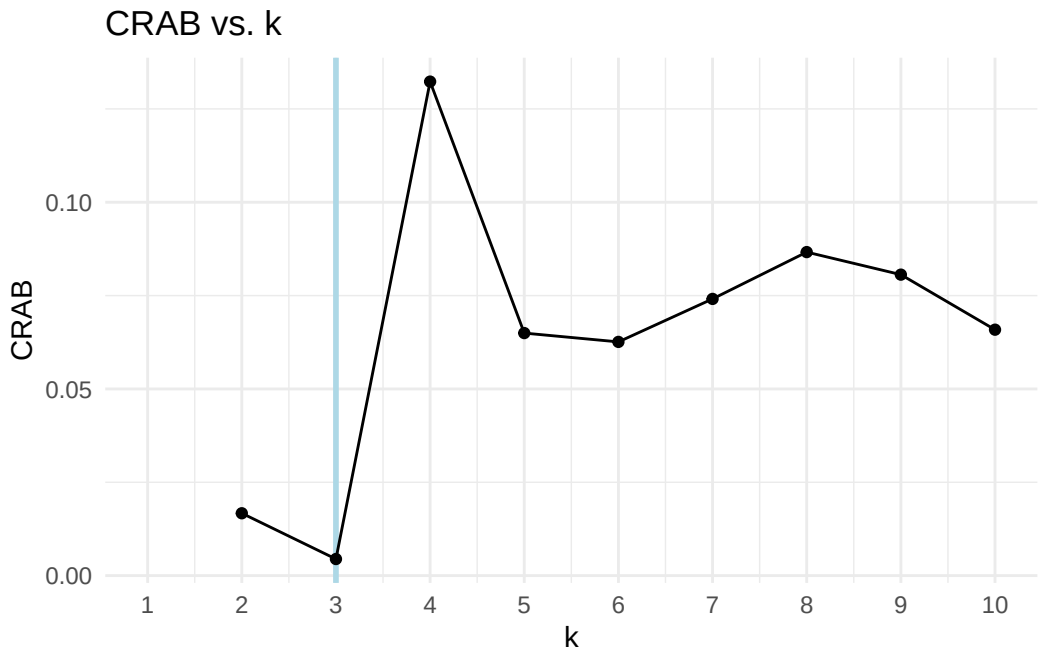
2.3 Known Data

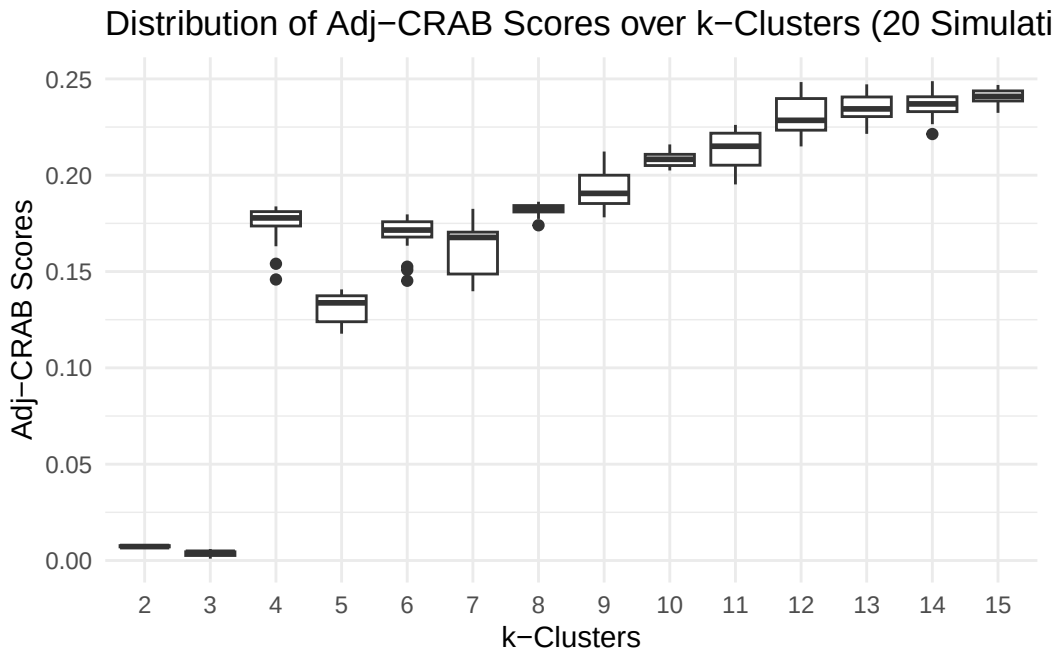
2.4 Performance on penguins dataset




Elbow Method on Example Dataset
WCSS
600
400
200
1
2
3
4
5
6
7
8
9
10
k


Silhouette Score vs. k
Silhouette
0.50
0.45
0.40
0.35
1
2
3
4
5
6
7
8
9
10
k


CRAB vs. k
CRAB
0.00
0.05
0.10
1
2
3
4
5
6
7
8
9
10
k


Distribution of Adj−CRAB Scores over k−Clusters (20 Simulati
Adj−CRAB Scores
0.00
0.05
0.10
0.15
0.20
0.25
2
3
4
5
6
7
8
9
10
11
12
13
14
15
k−Clusters

Chapter 3

COMPUTATIONAL DETAILS

The main principle of our algorithm works off of is the fact that subsamples of a dataset should still capture the overall clustering structure, which is also echoed in the paper from Ben-Hur (2002). This means that given two distinct points and their cluster assignments across resamples, the clustering assignments should be consistent. For example, two points that are far away from each other should consistently not be in the same cluster, and two points that are close together should consistently be in the same cluster, even across resamples.

3.1 Matrix Stacking

We now need a framework for the handling of resampling the matrices.

We define a "resample matrix" or "cluster assignment matrix" for each resample, where for an $n \times n$ resample matrix m, for every pairwise interaction between observation i and observation j we have a value of 1 in the matrix if they belong to the sample cluster, -1 if not, and 0 if either point was not in the resample.

Denoted another way, for two points i, j in the resample matrix, the element p_{ij} of the resample matrix is defined as:

$$p_{ij} = \begin{cases} 1, & \text{if points i, j belong to the same cluster} \\ -1, & \text{if points i, j do not belong to the same cluster} \\ 0, & \text{if either point i or point j are not in the resample} \\ \text{NA}, & \text{if pairwise element ij is located in the lower triangle or diagonal} \end{cases}$$

3.2 Algorithm Construction

After we have m matrices, we then reduce the matrices, taking the mean with respect to each index pair, ignoring zeroes. We then receive a final "mean matrix" of size n x n, where n is the number of observations in the dataset.

3.3 Score Construction

To illustrate the construction of the final score given this mean matrix, we use the penguins dataset in R with three known species and scaled bill length and flipper lengths.

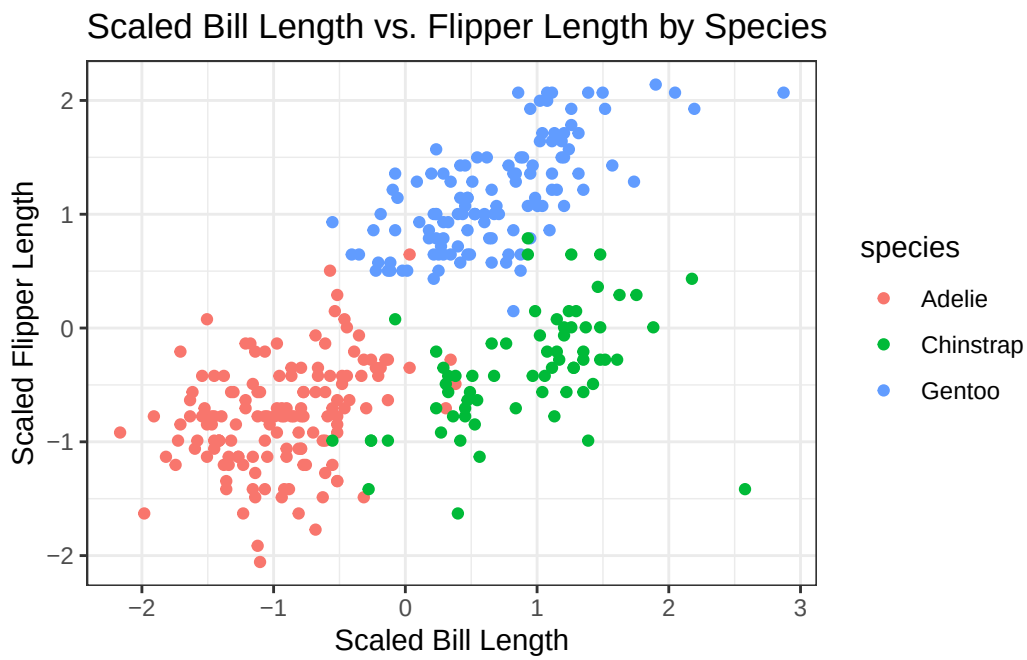


Figure 3.1: Plot of Penguin Species by Bill and Flipper Length

For the purposes of designing the scoring algorithm, we used this dataset with the three "known" clusters in order to fine-tune the behavior of the scores. Ideally, a scoring algorithm would output a lower score when the number of clusters input into the algorithm is not equal to three. Before defining this algorithm, it was helpful to first see the raw mean vector of values after the mean_matrix function, which simply averages the p_{ij} values over

all the resample matrices. Note that the values of ones and negative ones were removed, since a vast majority of points either consistently are in the same cluster or are consistently not in the same cluster.

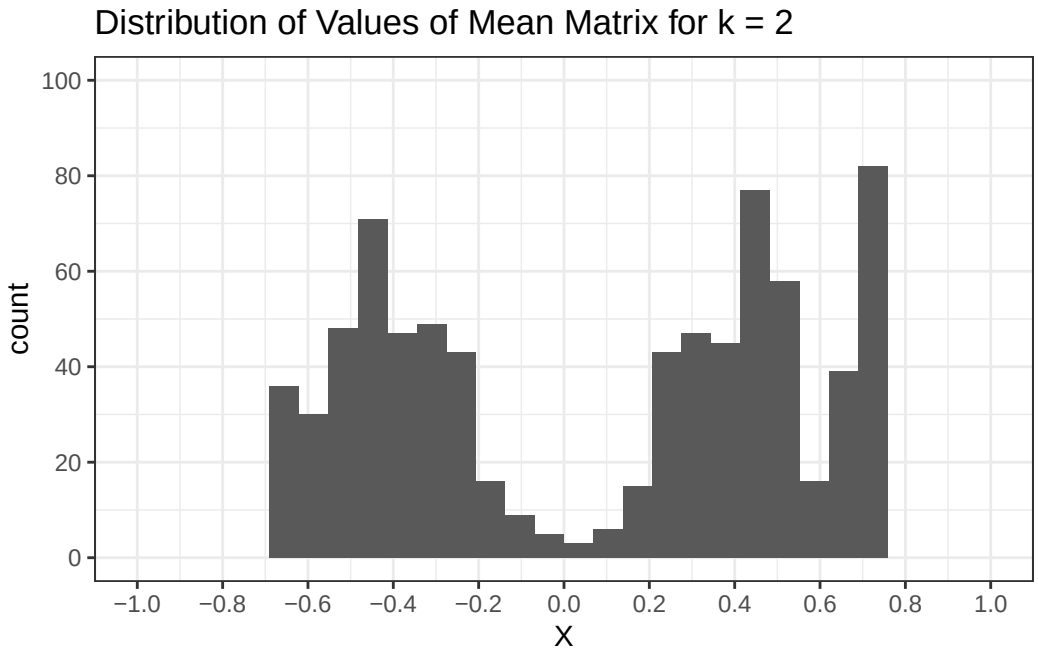


Figure 3.2: The distribution of values inside the mean matrix after running the algorithm on the penguins dataset, using K-Means and k = 2

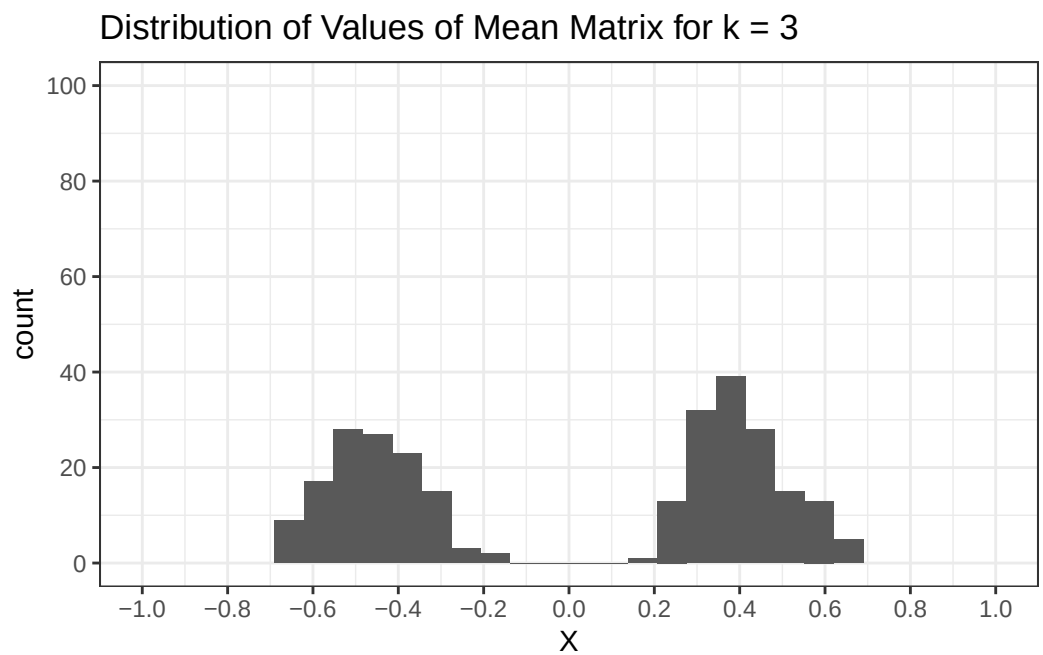


Figure 3.3: The distribution of values inside the mean matrix after running the algorithm on the penguins dataset, using K-Means and k = 3

We see that from the above figures that the distribution of values inside the mean matrix changes depending on the number of clusters set beforehand in the K-Means algorithm, albeit slightly. For k = 2, there are a small number of points around the value of 0, which indicates that these specific points are ones who are changing "allegiances" often, meaning that they switch from being "buddies" (being in the same cluster) and to being "rivals" (not being in the same cluster).

This informed our next direction of how to construct the scoring algorithm combining all of this information: Our algorithm should heavily penalize points that constantly change allegiances and penalize points that are consistent (whether consistently rivals or consistently buddies) less. Pairs of observations that are consistently in the same cluster would have an average p_{ij} value of 1 or close, and pairs of observations that are consistently not in the same cluster would have an average p_{ij} value of -1 or close. Then, pairs of observations that switch allegiances would have p_{ij} values close to 0.

To make a final score taking all this into account, we decided to take the absolute value of the vectorized mean matrix, which as a side effect would give consistent buddies and consistent rivals the same weight in our score. With this, we would obtain the sum of squared distances from 1, allowing us to penalize those pairs of observations that are inconsistent.

Our final scoring algorithm ends up as:

$\sum_i^n \sum_j^n (1-\bar{p}_{ij})^2$, where $\bar{p}_{ij}$ is the average of the p_{ij} values over all m resample matrices.

3.4 Point Contribution

3.5 Speed and Memory Scaling

3.5.1 Scaling with the Number of Observations

As mentioned previously in the paper, the CRAB algorithm involves the construction of m resample matrices of size $n \times n$, where n is the number of observations in the dataset. Predictably, the time complexity of the algorithm roughly scales by the order of $m \cdot n^2$, without taking into account the complexity of the resample matrix construction or the clustering algorithm itself. This is because the m $n \times n$ resample matrices are reduced to a single mean matrix for the calculation of the CRAB score.

With the k-Means algorithm, it has a time complexity that scales by the order of $n \cdot p \cdot q \cdot k$, where n is the number of observations, p is the number of dimensions, q is the number of iterations, and k is the number of clusters (Wang & Song, 2011). For our purposes we will only focus on the number of observations and number of dimensions due to the fact that the maximum number of iterations is capped at 10 and the number of clusters tested with the CRAB algorithm is small (less than 40). The k-Means algorithm is ran for each resample matrix as well.

Taking these time complexities into account, we can expect the entire algorithm to roughly

scale at $O(m \cdot (n^2 + np))$, where m is the number of resample matrices, n is the number of observations, and p is the number of dimensions of the dataset.

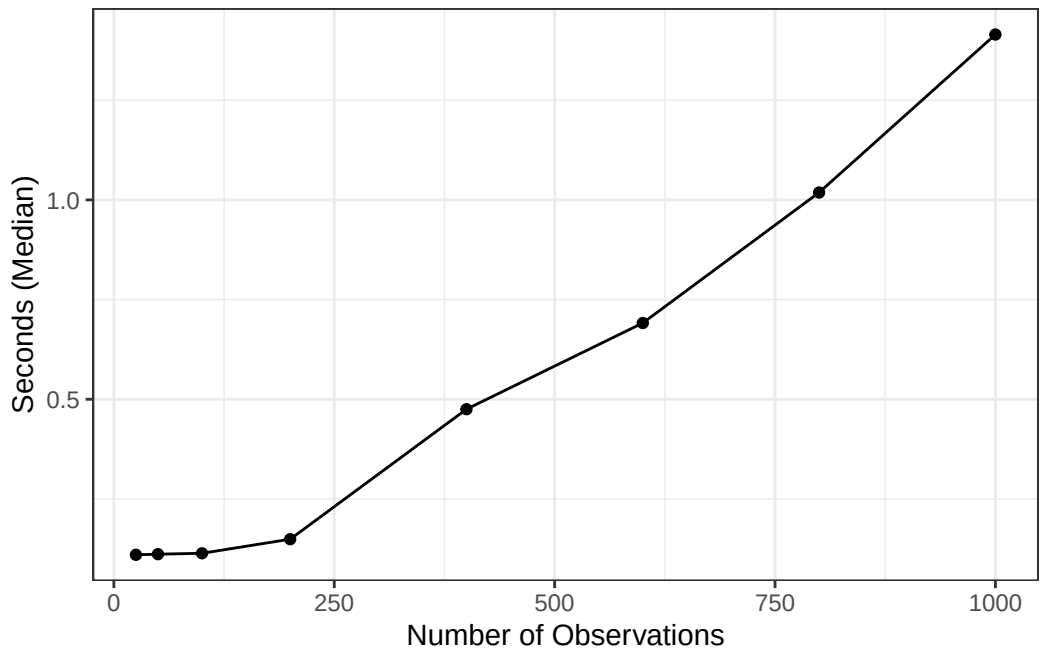


Figure 3.4: Speed of the CRAB Algorithm Quadratically Increases as Number of Observations Increases

As for memory scaling, due to the fact that m $n \times n$ resample matrices are constructed, we can also expect memory usage to scale quadratically with respect to the number of observations, regardless of the clustering algorithm used. This behavior can be seen when testing the CRAB algorithm with k-Means using the **bench::mark** function. It is important to note that a single score was computed from the results of a single call of the CRAB algorithm on a specific k of three using 50 resamples on a random p-dimensional Normal distribution with n observations; in actuality to compare the viability of each k the CRAB algorithm would need to be run k times, once for each k of interest. The full details and code of the implementation are available on the GitHub repository under **analyses/speed_memory_scaling.qmd**.

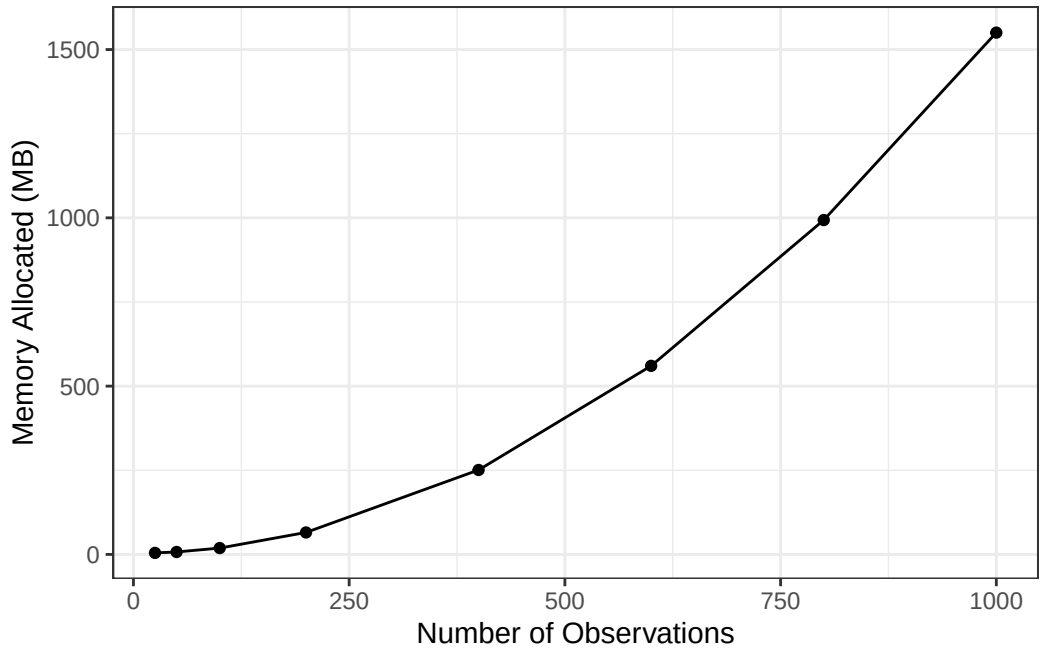


Figure 3.5: Memory of the CRAB Algorithm Quadratically Increases as Number of Observations Increases

This, of course, is not ideal, but does guide possible future work improving the algorithm. At a large n the resample matrices become extremely sparse, leading to possible memory savings if treated as such. However, the fact remains that the algorithm relies on the storage of all m resample matrices in some capacity.

Unlike when changing the number of observations, increasing the number of dimensions should not have a large impact on the speed and memory performance of the CRAB algorithm itself. Since we rely on the clustering assignments output from the clustering algorithm for the CRAB implementation, the actual differences in time and memory usage come almost completely from the clustering algorithm used. As stated earlier, the time complexity of the k-Means algorithm scales linearly, scaling with np.

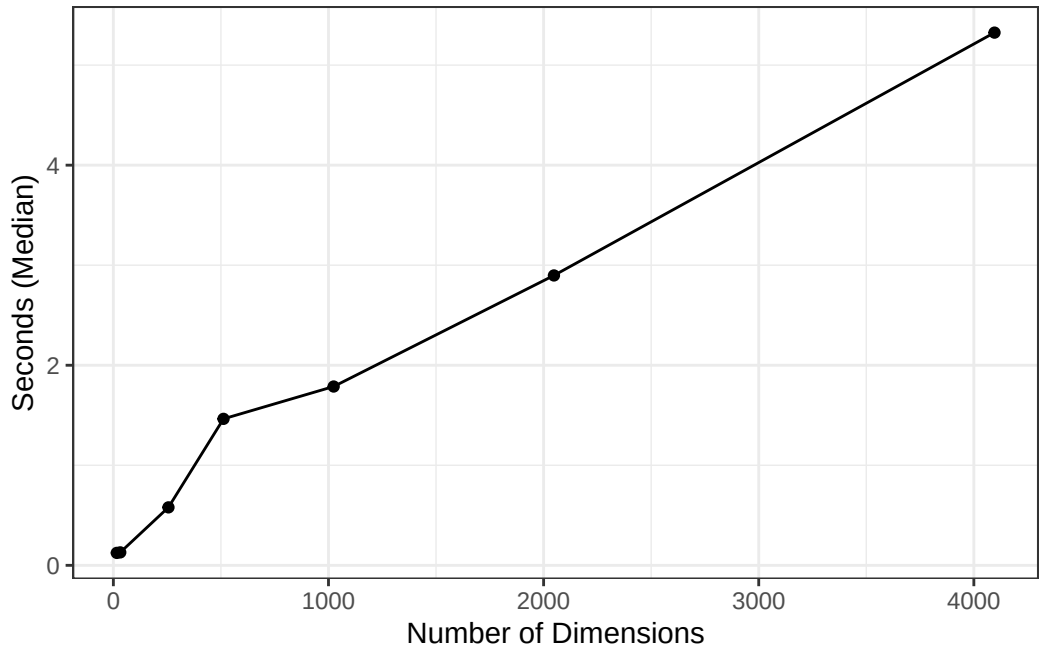


Figure 3.6: Speed of the CRAB Algorithm Increasing Linearly as Number of Dimensions Increases

The k-Means algorithm has a space complexity of $O((n + k) \cdot p)$, where p is the number of dimensions of the dataset (Ikotun et al., 2023). When testing for the memory usage, we see this linear behavior as well.

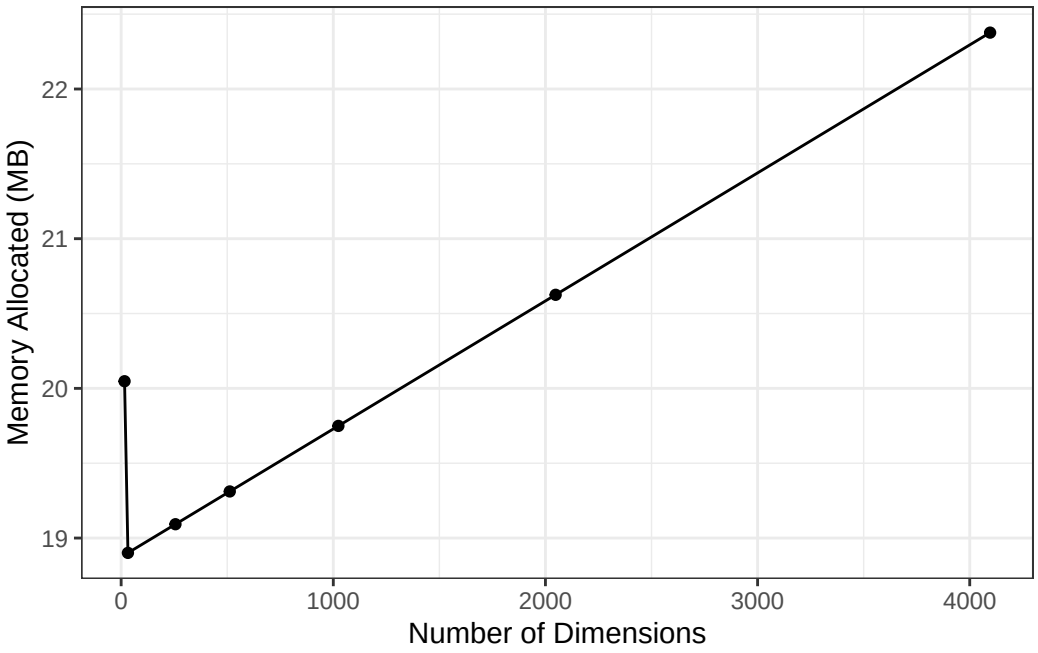


Figure 3.7: Memory of the CRAB Algorithm Increasing Linearly as Number of Dimensions Increases

Chapter 4

NULL DISTRIBUTION

4.1 Null Distribution of Scores

It is also interesting information to see what the distribution of scores looks like when there are no clusters in the dataset.

The hope was that our scores can be adjusted depending on the behavior of the CRAB scores when there are in actuality no clusters (or in this case one cluster). Based off the behavior of this "null distribution", we can adjust the CRAB score.

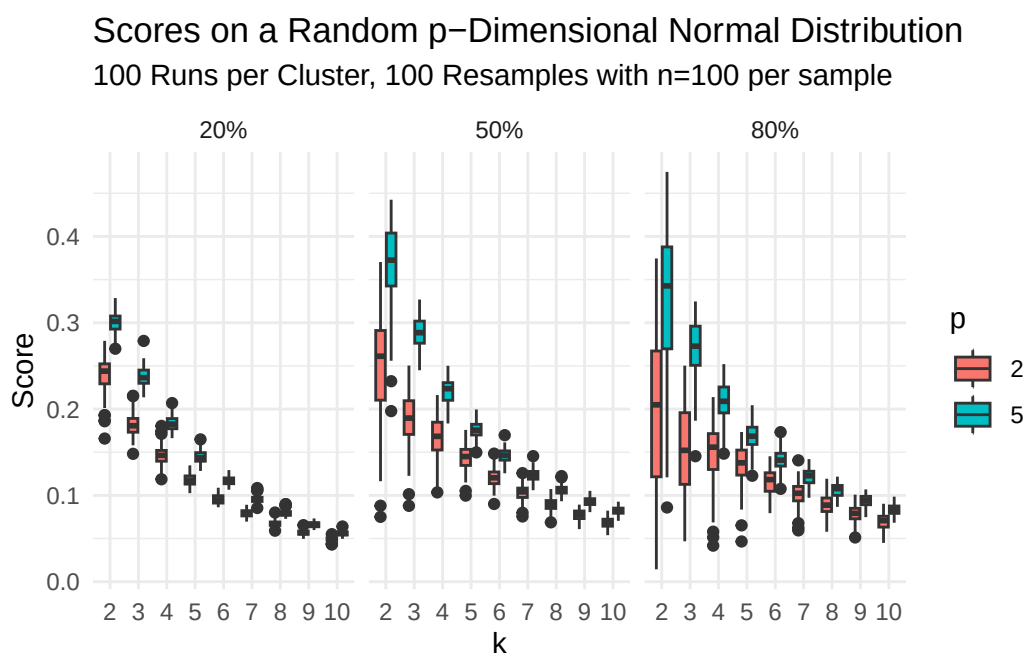


Figure 4.1: Distribution of Scores on a Random p-Dimensional Normal Distribution, Faceted by Resample %

We see in the above figure that the resample proportion has a significant impact on the distribution of scores, where the variability and mean of scores over many simulation runs

is dependent on the resample proportion. The scores also tend to be higher, scaling with p-dimensions, with a decreasing score distribution as k increases. Notably, all these scores look symmetrically distribution around their mean, with respect to each k.

4.2 Null Distribution over Resamples

```
`summarise()` has grouped output by 'factor(k)'. You can override using the
`.groups` argument.
```

Table 4.1: Variances for 20% Resample Calculation

factor(k)	factor(prop_resample)	var
2	0.2	0.1347830
3	0.2	0.1471547
4	0.2	0.1285054
5	0.2	0.1161971
6	0.2	0.1133949

4.3 Adj-CRAB Score

Chapter 5

CONCLUSION

On the penguins dataset, we use 100 resamples and a resample proportion of 80% wih the k-Means algorithm and a given k, using again the scaled bill length and flipper lengths in our dataset.

(i guess this should also go in the analysis portion??)

REFERENCES

Ben-Hur, A., Elisseeff, A., & Guyon, I. (2001). A stability based method for discovering structure in clustered data. *Biocomputing 2002*. Kauai, Hawaii, USA: WORLD SCIENTIFIC. https://doi.org/10.1142/9789812799623_0002

Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226–231. Portland, Oregon: AAAI Press.

Hvitfeldt, E., & Bodwin, K. (2025). *Tidyclust: A common API to clustering*. https://doi.org/10.32614/CRAN.package.tidyclust

Ikotun, A. M., Ezugwu, A. E., Abualigah, L., Abuhaija, B., & Heming, J. (2023). K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data. *Information Sciences*, *622*, 178–210. https://doi.org/10.1016/j.ins.2022.11.139

Jaeger, A., & Banks, D. (2023). Cluster analysis: A modern statistical review. *WIREs Computational Statistics*, *15*(3). https://doi.org/10.1002/wics.1597

Kuhn, M., & Wickham, H. (2020). *Tidymodels: A collection of packages for modeling and machine learning using tidyverse principles.* Retrieved from https://www.tidymodels.org

R Core Team. (2025). *R: A language and environment for statistical computing*. Vienna, Austria: R Foundation for Statistical Computing. Retrieved from https://www.R-project.org/

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, *20*, 53–65. https://doi.org/https://doi.org/10.1016/0377-0427(87)90125-7

Schubert, E. (2023). Stop using the elbow criterion for k-means and how to choose the number of clusters instead. *ACM SIGKDD Explorations Newsletter*, *25*(1), 36–42. https://doi.org/10.1145/3606274.3606278

Thorndike, R. L. (1953). Who belongs in the family? *Psychometrika*, *18*(4), 267–276. https://doi.org/10.1007/BF02289263

Wang, H., & Song, M. (2011). Ckmeans.1d.dp: Optimal k-means Clustering in One Dimension by Dynamic Programming. *The R Journal*, *3*(2), 29–33. Retrieved from https://pmc.ncbi.nlm.nih.gov/articles/PMC5148156/

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., … Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, *4*(43), 1686. https://doi.org/10.21105/joss.01686

APPENDICES

Appendix A

RESAMPLE MATRIX

```
resample_function <- function(data = data,
                              formula = ~ .,
                              k = 3,
                              number_of_resamples = 15,
                              proportion_resample = 0.9,
                              starting_seed = 599,
                              algorithm = "kmeans") {
  data <- data |>
    drop_na()
  data$index <- 1:nrow(data)
  results <- list()

  # For loop
  results <- foreach(i = 1:number_of_resamples,
                     .packages = c("tidyclust",
                                   "dplyr",
                                   "tidymodels")) %dopar% {
    # Reproducibility over parallel
    set.seed(starting_seed + i)
    result_matrix <- matrix(0, nrow = nrow(data),
```

```
                          ncol = nrow(data))


random_sample <- data |>
  filter(index %in% sample(index,
                           proportion_resample * nrow(data)))


if (algorithm == "kmeans") {
  cluster_assigned <- k_means(num_clusters = k) |>
    fit({{formula}},
        data = random_sample |>
          select(-index))
} else if (algorithm == "hier_clust") {
  cluster_assigned <- hier_clust(num_clusters = k) |>
    fit({{formula}},
        data = random_sample |>
          select(-index))
} else {
  stop("Algorithm is not supported. Please select
       one of (kmeans, hier_clust)")
}



intermediate <- data.frame(random_sample$index,
                           extract_cluster_assignment(
                             cluster_assigned
                             ) |>
                             mutate(.cluster = as.character(
```

```
                                    .cluster)
                                    ),
                                stringsAsFactors = FALSE)
    colnames(intermediate) <- c("index", "cluster")

    for (c in unique(intermediate$cluster)) {
      idx <- intermediate[intermediate$cluster == c, ]$index

      # Check that the index list is not length 1 for
      # purposes of n > m for combn
      # This also fixes a bug in the
      # previous code where idx was wrongly
      # interpreted as single numerical
      # value as param inside combn
      if (length(idx) > 1) {
        idx <- sort(idx)
        ones <- t(combn(idx, 2))
        result_matrix[cbind(ones[, 1], ones[, 2])] <- 1
      }

      neg_one_idx <- expand.grid(idx, setdiff(random_sample$index, idx))
      result_matrix[cbind(neg_one_idx[, 1], neg_one_idx[, 2])] <- -1
    }
    result_matrix[lower.tri(result_matrix, diag = TRUE)] <- NA
    result_matrix
  }
return(results)
```

```
}
```

Appendix B

MEAN MATRIX

```r
mean_matrix <- function(list_of_matrices = list_of_matrices) {
  # Absolute value the entire matrix, for all matrices
  absolute_sum <- lapply(list_of_matrices, abs)

  # Get the sum of absolute values in the final matrix
  absolute_final <- absolute_sum |>
    reduce(`+`)

  # Get the sum of values in the final matrix
  numerator <- list_of_matrices |>
    reduce(`+`)

  # Returns the matrix of zero-removed means.
  # The zero-removed mean is the sum over
  # the (length - the number of zeroes).
  # The length - the number of zeroes is
  # equivalent to the sum over absolute values
  return(numerator / absolute_final)
}
```

Appendix C

METRIC CALCULATION

```r
squared_distance_from_one <- function(mean_matrix = mean_matrix) {
  # Ignore the order of the elements in matrix
  res_vec <- as.vector(mean_matrix)

  # Absolute value and remove NAs
  res_vec <- abs(res_vec[!is.na(res_vec)])

  # Take average squared distance from one
  return(mean((1 - res_vec)^2))
}
```

Appendix D

POINT CONTRIBUTION FUNCTION

```
point_contribution <- function(data, mean_matrix) {
  # Given the original data (the dataset inputted into the resample_matrices funct
  # and the mean_matrix, returns each observation's contribution to the CRAB Score
  # We don't necessarily require the second arg to be the mean matrix, for example
  # it could be resample matrix j.
  # This is returned as an (index, score) dataframe.

  # Get a list of lists containing index and index "CRAB" score
  scores <- lapply(c(1:nrow(data)), function(i) {
    full_vec <- getPointVecHelper(i, mean_matrix)
    singlePointScore <- mean((1 - full_vec)^2)
    return(list(index = i, singlePointScore = singlePointScore))
  })
  final_df <- do.call(rbind.data.frame, scores)
  total_score <- mean(final_df$singlePointScore)

  final_df <- final_df |>
    mutate(singlePointContribution = singlePointScore / total_score)

  return(final_df)
}
```

Appendix E

GET POINT VECTOR HELPER

```
getPointVecHelper <- function(i, mean_matrix) {
  # Given a point i and the mean matrix, should
  # return vector of all values in the mean matrix corresponding to obs i
  # after removing all NAs and then taking absolute value

  row <- mean_matrix[i, ]
  col <- mean_matrix[, i]

  # We don't care about the order, just the values themselves
  res_vec <- c(row, col)
  res_vec <- abs(res_vec[!is.na(res_vec)])
  return(res_vec)

}
```

Appendix F

CRAB SCORE ACROSS K CLUSTERS

```
crab_k_across <- function(data = data,
                          c_vec = c_vec,
                          formula = ~ .,
                          number_of_resamples = 80,
                          proportion_resample = 0.8,
                          starting_seed = 599,
                          algorithm = "kmeans") {
  # Calls the resampling function across the dataset, with the parameters
  # passed to the resampling function.

  # Returns a dataframe with k rows (one for each element in the c_vec cluster vec
  # and a value for the cRab score after running the cRab algorithm on the dataset

  result <- lapply(c(c_vec), function(k) {
    resample_matrices <- resample_function(
      data = data,
      formula = formula,
      k = k,
      number_of_resamples = number_of_resamples,
      proportion_resample = proportion_resample,
      starting_seed = starting_seed,
      algorithm = algorithm
```

```
      )

    m_matrix <- mean_matrix(resample_matrices)
    score <- squared_distance_from_one(m_matrix)

    return(list(k = k,
                cRab = score))
  })
  return(dplyr::bind_rows(result))
}
```

```
# resample_matrices <- resample_function(penguins,
#                                        number_of_resamples = 40,
#                                        proportion_resample = 0.8,
#                                        starting_seed = 100)
# m_matrix <- mean_matrix(resample_matrices)
#
#
# d <- point_contribution(penguins, m_matrix)
#
# test <- lapply(resample_matrices,
#        point_contribution,
#        data = penguins)
#
# mean_var_single <- do.call(rbind.data.frame, test) |>
#   group_by(index) |>
#   summarize(mean = mean(singlePointScore),
```

```
#             var = sd(singlePointScore)) |>
#   arrange(desc(var))
```